Lab_Spark SQL with Temporary Views & PySpark UDF Implementation

Lab Name:

Spark SQL with Temporary Views & PySpark UDF Implementation

Lab Summary:

In this lab, you will:

- Execute Spark SQL query inside PySpark notebook
- Load data from Silver layer table
- Convert DataFrame into Temporary View
- Apply `CASE WHEN` logic using Spark SQL
- Create a custom PySpark UDF
- Apply UDF to DataFrame column
- Generate a new derived column using UDF

This lab demonstrates how to combine **Spark SQL + PySpark UDF** in a single workflow.

Prerequisites:

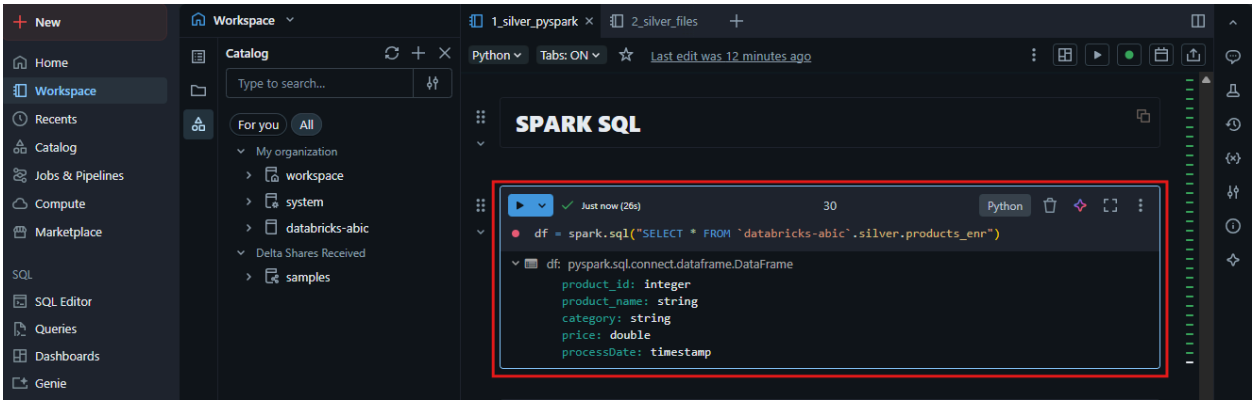
- Unity Catalog enabled
- `databricks-abic` catalog available
- `silver.products_enr` table created
- Compute cluster running

Steps to Complete the Lab

Task 1: Spark SQL Execution

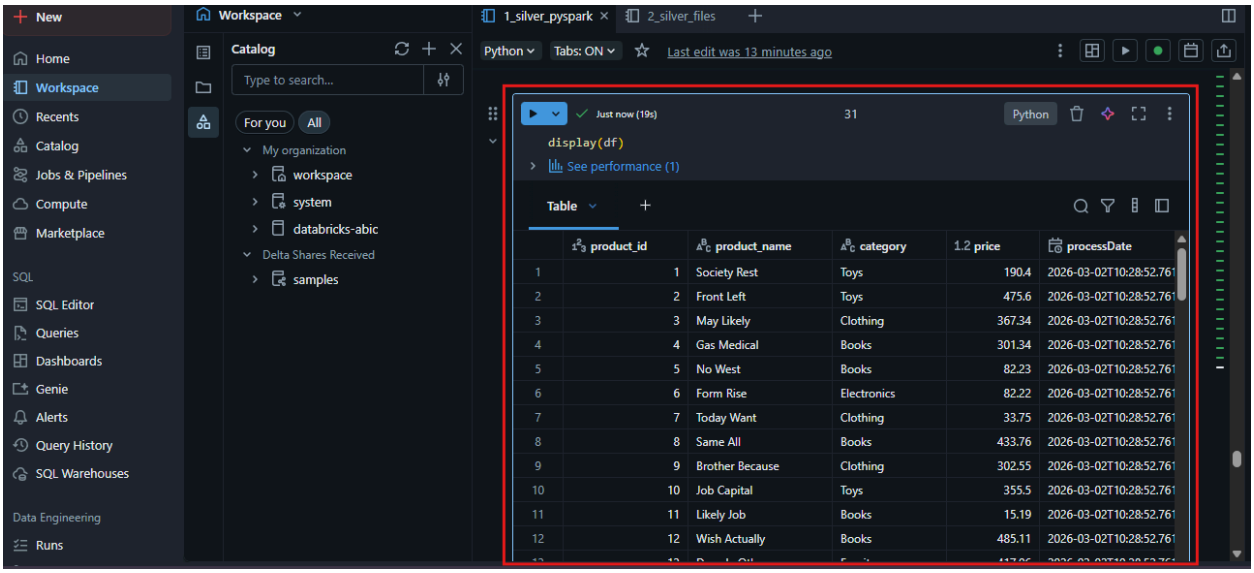
Step 1.1: Read Silver Table Using spark.sql()

```
df = spark.sql("SELECT * FROM `databricks-abic`.silver.products_enr")
```



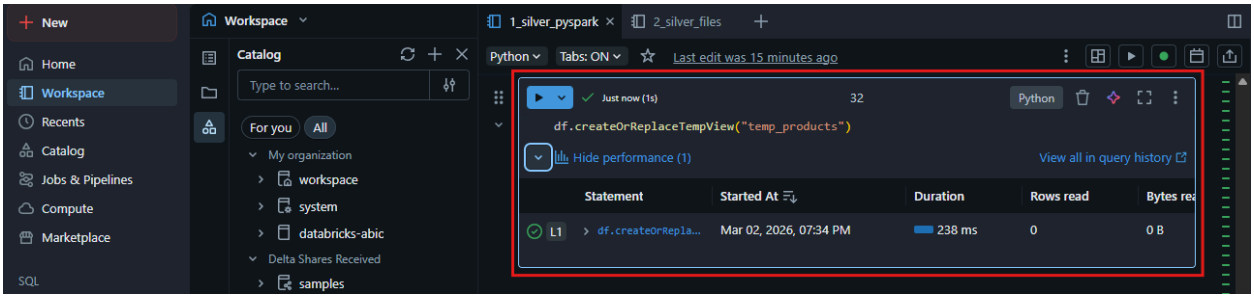
Step 1.2: Display Data

```
display(df)
```



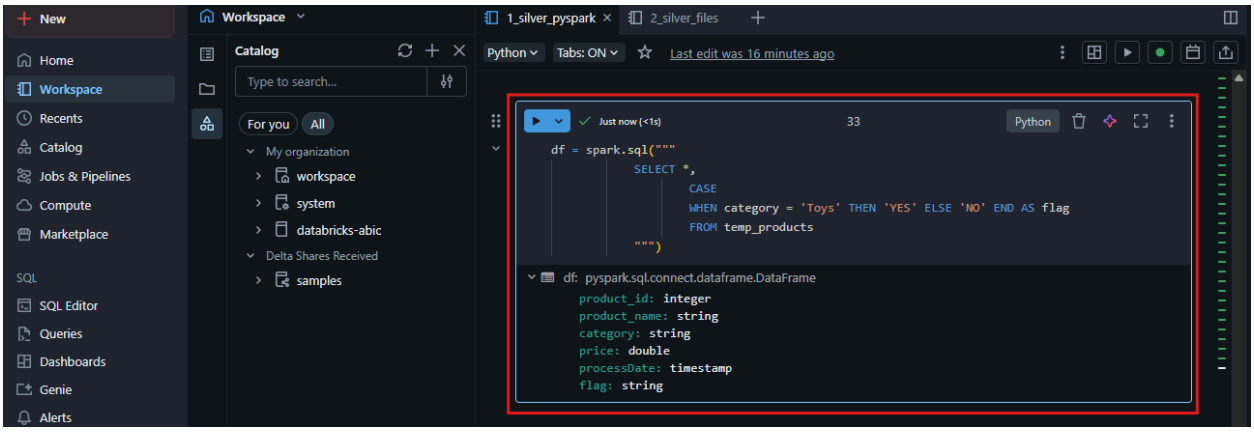
Step 1.3: Convert DataFrame to Temporary View

```
df.createOrReplaceTempView("temp_products")
```



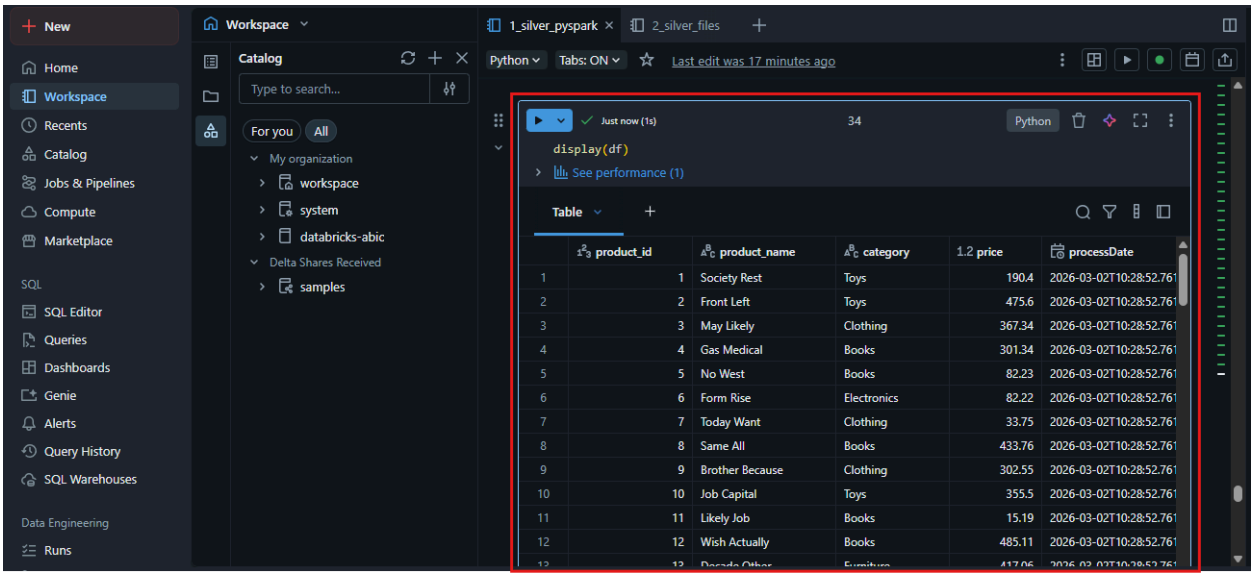
Step 1.4: Apply CASE WHEN Using Spark SQL

```
df = spark.sql("""
            SELECT *,
                    CASE
                    WHEN category = 'Toys' THEN 'YES'
                    ELSE 'NO'
                    END AS flag
            FROM temp_products
            """)
```



Step 1.5: Display Updated DataFrame

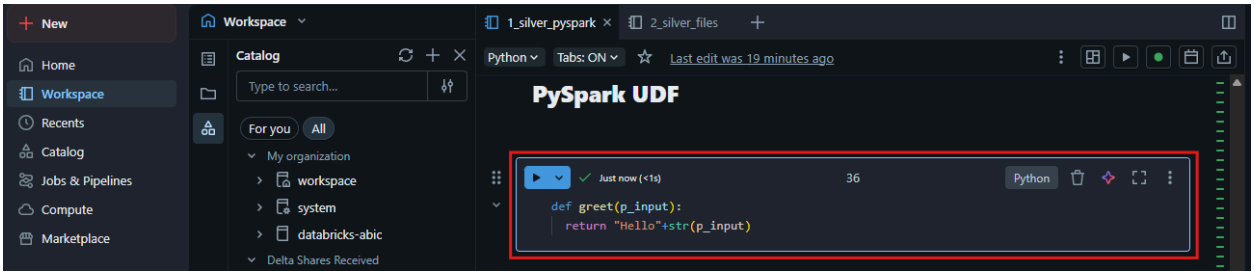
```
display(df)
```



Task 2: PySpark UDF Implementation

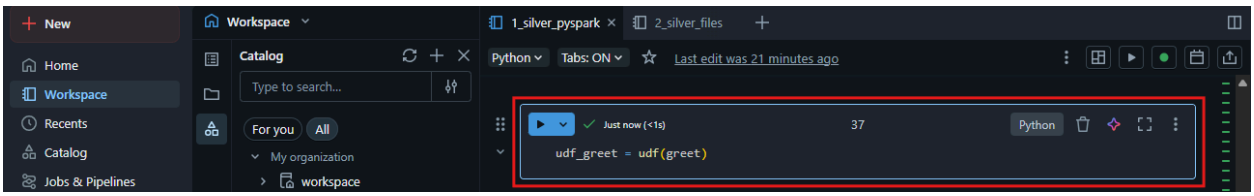
Step 2.1: Define Python Function

```
def greet(p_input):
    return "Hello" + str(p_input)
```



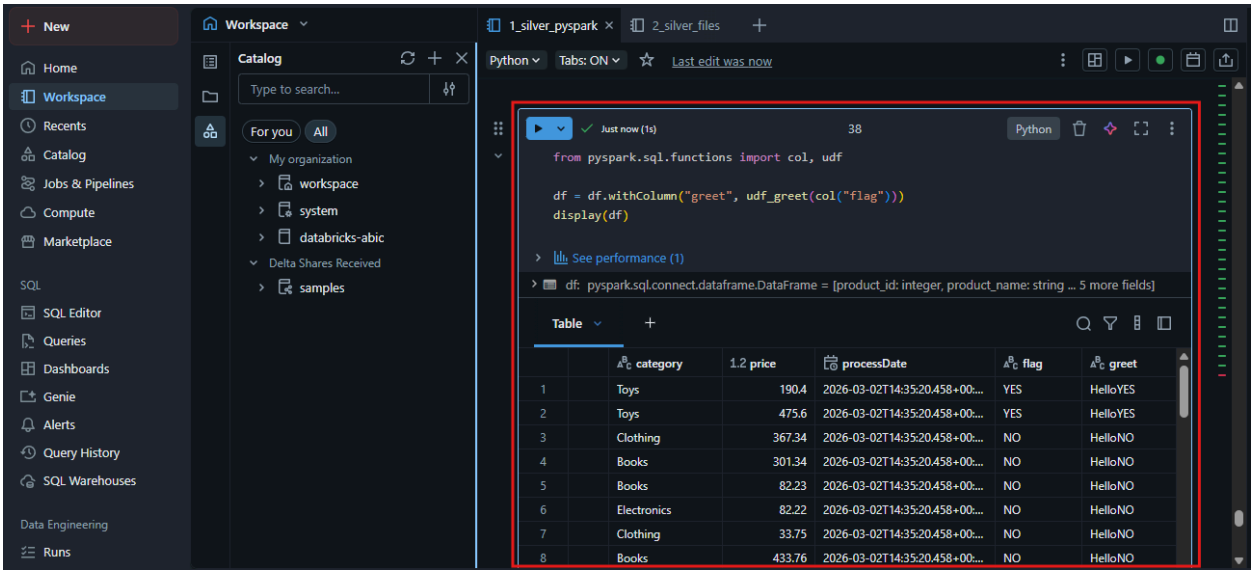
Step 2.2: Convert Python Function to UDF

```
udf_greet = udf(greet)
```



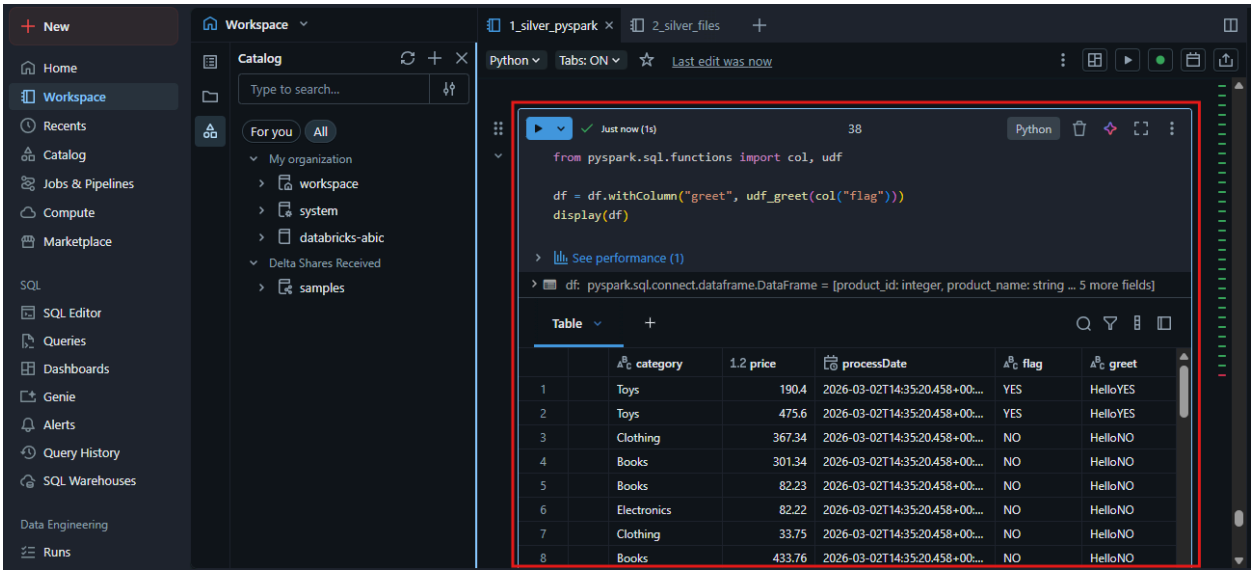
Step 2.3: Apply UDF to DataFrame Column

```
df = df.withColumn("greet", udf_greet(col("flag")))
display(df)
```



Step 2.5: Display Final Output

```
display(df)
```



Outcome:

After completing this lab, you:

✔️ Queried table using Spark SQL

✔️ Created Temporary View

✔️ Applied `CASE WHEN` logic

✔️ Created a custom Python function

✔️ Converted function into PySpark UDF

✔️ Applied UDF to DataFrame column

✔️ Generated new derived column